



UNIVERSIDADE FEDERAL DE ITAJUBÁ

Algoritmos e Estrutura de Dados I

CTC001

Fila de Prioridade e TAD Heap

Vanessa Cristina Oliveira de Souza



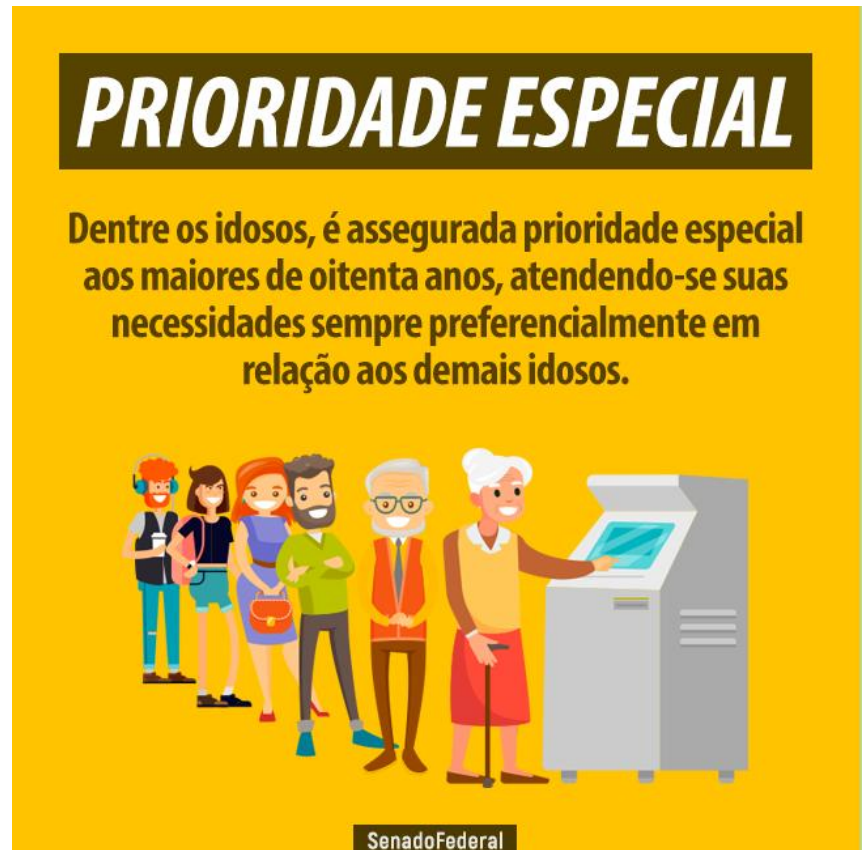
Filas de Prioridade

- ▶ O TAD fila, estudado no início do curso, apresenta como característica principal o fato de seus mecanismos de inserção e remoção atuarem apenas em suas extremidades.
- ▶ O problema com filas é que não podemos diferenciar, para tratamento privilegiado, elementos que porventura estejam nelas armazenados



Filas de Prioridade

- ▶ Nesse caso, como poderíamos modelar a estrutura fila?
- ▶ considerando que temos uma única fila





Filas de Prioridade

- ▶ Dada a fila abaixo, podemos dizer que existe uma fila dentro da fila.
 - ▶ As duas primeiras pessoas da fila são prioridade.
 - ▶ As demais estão na fila por ordem de chegada.
- ▶ Se chegar uma pessoa que não está na lista de prioridade, onde ela deve ser inserida?
- ▶ E se chegar uma pessoa com prioridade? Onde ela deve ser inserida?
- ▶ E onde acontece a saída da fila? (remoção)





Filas de Prioridade

- ▶ Uma forma de tratar de modo diferente os diferentes elementos armazenados numa fila surge com a criação de filas de prioridade
- ▶ Numa fila de prioridade um **valor de prioridade** é atribuído a cada elemento e as operações de inserção e remoção passam a ser feitas em função dessa prioridade





Filas de Prioridade

- ▶ Uma fila de prioridade é uma estrutura de dados abstrata que funciona como uma fila normal, mas cada elemento tem uma prioridade associada.
- ▶ Principais Aplicações:
 - ▶ Eventos processados em ordem cronológica
 - ▶ Escalonamento de processos em sistemas computacionais
 - ▶ Sistemas Operacionais: Usados no escalonamento de tarefas (*process scheduling*). O processo com a maior prioridade (ou o menor tempo de execução restante) é processado primeiro.
 - ▶ Computações numéricas (maiores erros são tratados primeiro)
 - ▶ Implementação de algoritmos de compressão
 - ▶ Algoritmo de Huffman Coding





Filas de Prioridade

- ▶ Embora tenham sido criadas a partir do tipo fila, as filas de prioridade podem ser implementadas a partir de duas estruturas básicas:
 - ▶ Listas (preferencialmente ordenadas)
 - ▶ Árvores (chamadas de heaps)
- ▶ Nesse curso vamos estudar a implementação de fila de prioridade utilizando **ÁRVORES HEAPS**.



A vertical blue bar is located on the left side of the slide, partially enclosed by a thin white rectangular border.

Árvores HEAP



Árvores HEAP

- ▶ Em inglês, HEAP significa “amontoadado” e essa foi a ideia por trás do nome. Um amontoadado hierárquico.
- ▶ A estrutura de dados **Heap** e o algoritmo **Heapsort** foram introduzidos e definidos no mesmo trabalho original de **J.W.J. Williams** em 1964.
- ▶ Heapsort é um algoritmo de ordenação que usa a estrutura de HEAP para ordenar um conjunto de elementos.





Árvores HEAP

- ▶ A estrutura de dados HEAP é:
 - ▶ Uma árvore binária onde cada elemento tem uma prioridade
 - ▶ É uma árvore quase-completa da esquerda para a direita
 - ▶ Se o último nível não estiver totalmente preenchido, ele deve ser preenchido da esquerda para a direita sem lacunas.
 - ▶ É uma árvore que precisa manter uma propriedade de ordem
 - ▶ Heap de Máximo
 - ▶ Heap de Mínimo

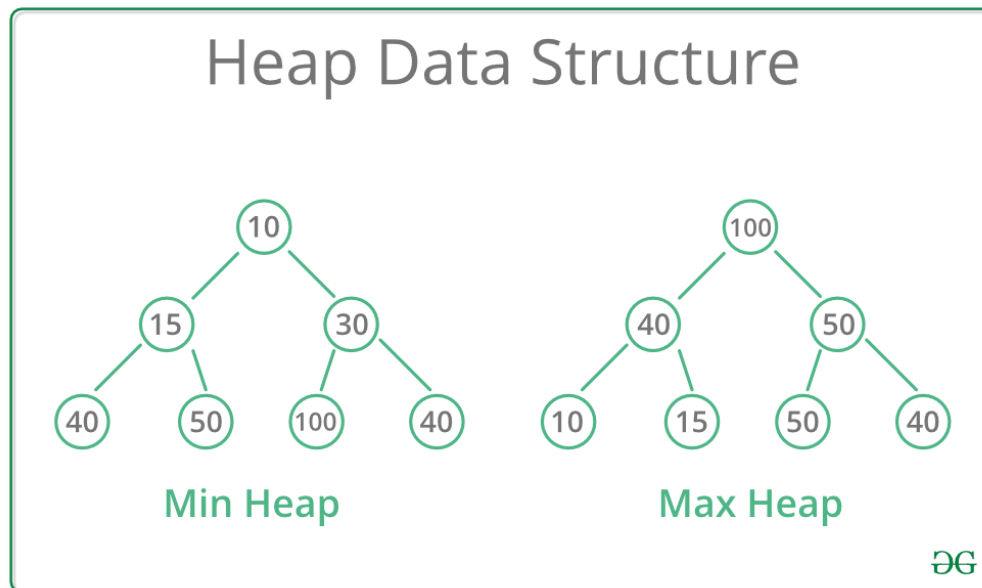
Um HEAP é uma árvore binária completa em ordem heap.





Árvores HEAP

- ▶ Uma árvore está em ordem **max-heap** se a chave em cada nó é maior ou igual às chaves dos seus filhos.
- ▶ De forma análoga, uma árvore está em ordem **min-heap** se a chave em cada nó é menor ou igual às chaves dos seus filhos.

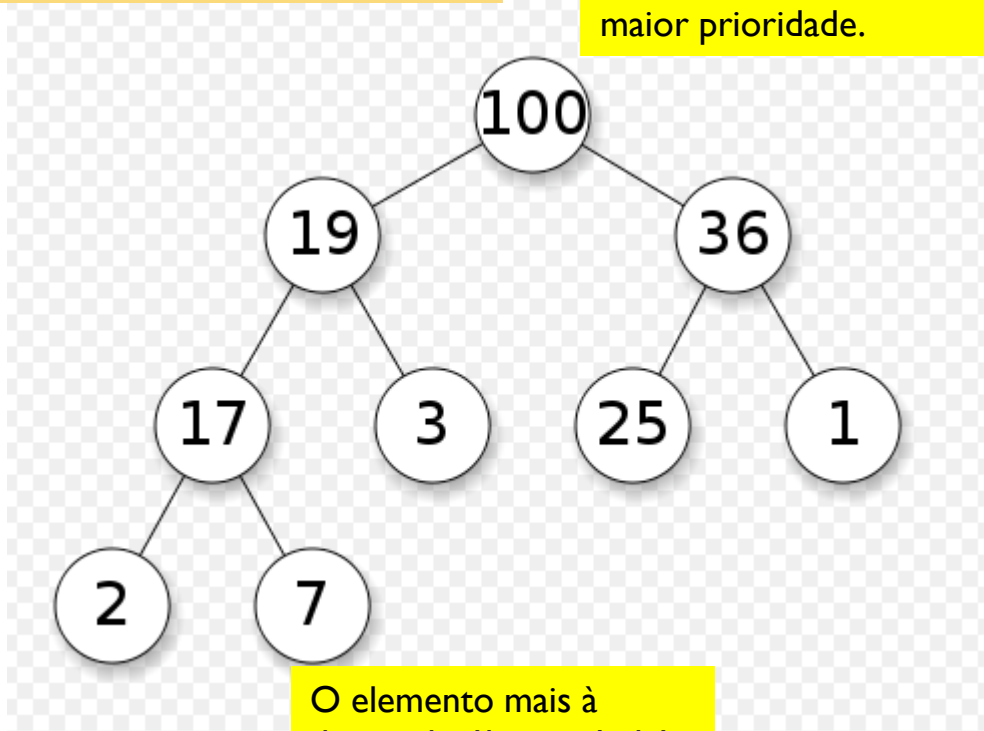




Árvores HEAP

**Essa é uma HEAP de
Máximo**

A raiz é o elemento com
maior prioridade.



O elemento mais à
direita do último nível é
o último elemento da
HEAP.

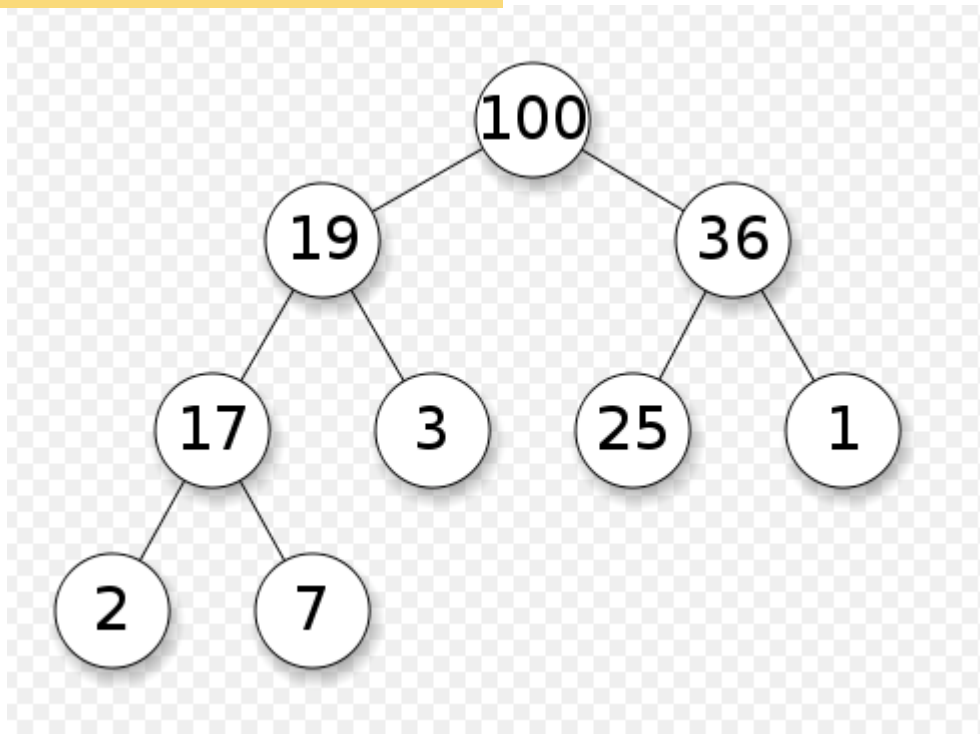
- ▶ Se fosse uma fila, a raiz (100) seria o próximo elemento a sair da fila
- ▶ O elemento 7 seria o último elemento da HEAP
 - ▶ Não necessariamente o último a ser processado
- ▶ O processamento dos dados se dá pela remoção e sempre é removido o elemento de maior prioridade na árvore.





Árvores HEAP

**Essa é uma HEAP de
Máximo**



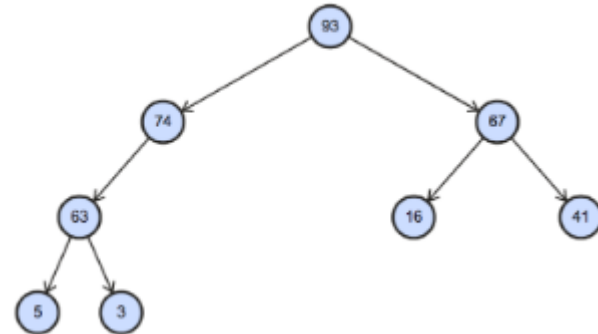
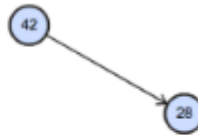
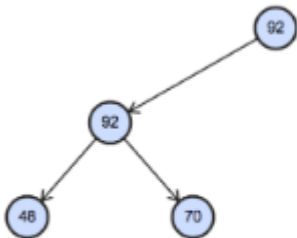
- ▶ Cada pai tem prioridade maior (max-heap) ou menor (min-heap) que seus filhos.
- ▶ Mas não existe ordem garantida entre elementos de níveis diferentes que não estejam na mesma subárvore.





Árvores HEAP

Exemplos de Árvores que não são HEAP

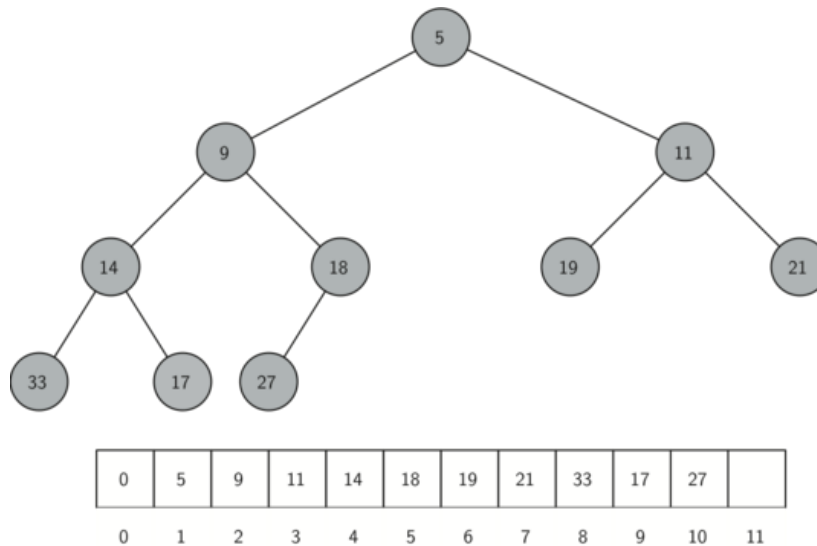




Árvores HEAP

Representação

- ▶ A forma mais comum e otimizada de representar uma HEAP é por meio de vetores.
- ▶ Representação mais simples, sem encadeamento
- ▶ É necessário saber a quantidade máxima de elementos à priori



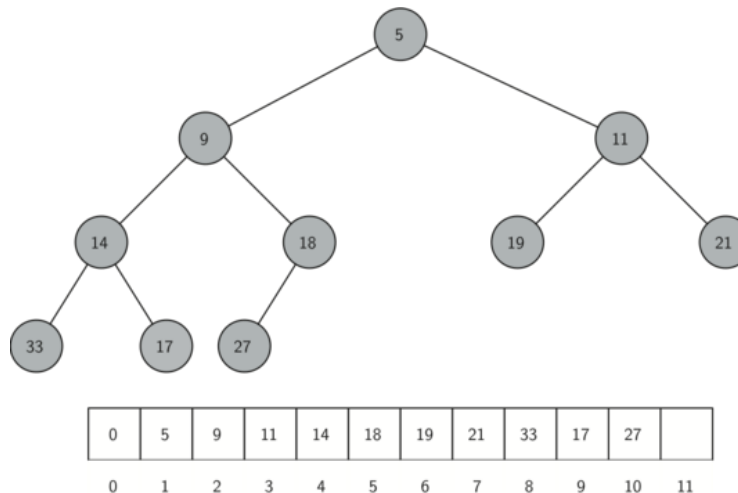
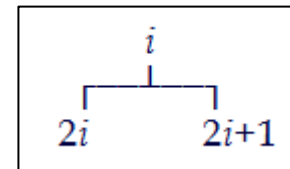


Árvores HEAP

Representação

► Regras:

- A posição 0 do vetor é ignorada
- A posição 1 guarda a raiz da árvore
- Dado um nó que está na posição i do vetor
 - O filho à esquerda está na posição $(2 * i)$
 - O filho à direita está na posição $(2 * i + 1)$



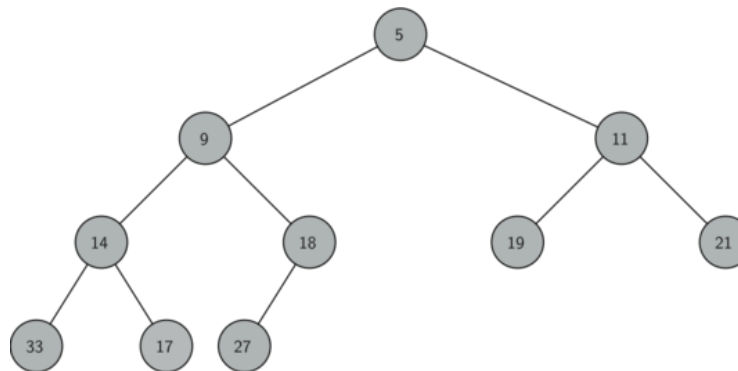


Árvores HEAP

Representação

► Regras:

- A posição 0 do vetor é ignorada
- A posição 1 guarda a raiz da árvore
- Dado um nó que está na posição i do vetor
 - O filho à esquerda está na posição $(2 * i)$
 - O filho à direita está na posição $(2 * i + 1)$



0	5	9	11	14	18	19	21	33	17	27	
0	1	2	3	4	5	6	7	8	9	10	11

tail = 10

Existe uma variável chamada *tail* que guarda a posição do último elemento da HEAP no vetor

Se a posição do filho é maior que *tail*, significa que não existe o filho.

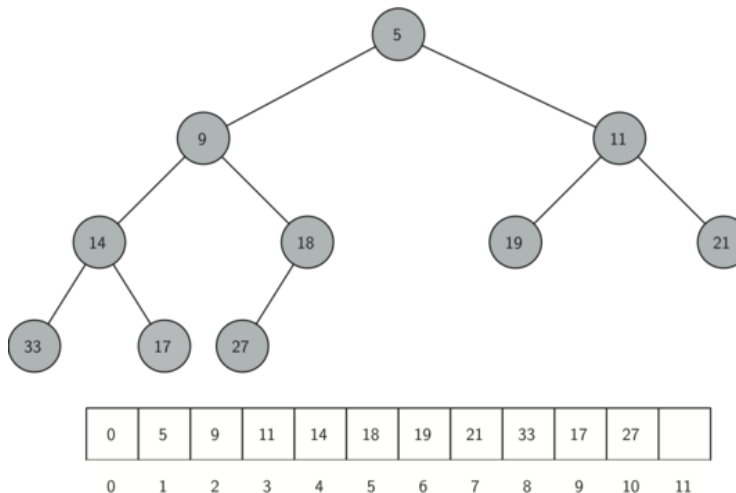


Árvores HEAP

Representação

► Regras:

- A posição 0 do vetor é ignorada
- A posição 1 guarda a raiz da árvore
- Dado um nó que está na posição i do vetor
 - O seu pai estará na posição $\lfloor i/2 \rfloor$
 - A raiz não tem pai



- ▶ O HEAP é uma fila e segue os mesmos conceitos de fila
 - ▶ Inserir no final da fila
 - ▶ Remover do começo da fila
 - ▶ Não há busca
- ▶ No entanto, HEAP é uma fila de **prioridade** e a ordenação (max ou min) precisa ser sempre garantida.
- ▶ Assim, as operações de inserir e remover são seguidas de uma operação que garante a ordenação da árvore.

A vertical blue bar is located on the left side of the slide, partially enclosed by a thin white rectangular border.

TAD HEAP

INSERÇÃO



Árvores HEAP

TAD - Inserção

► Considere os elementos:

► 33 70 66 28 39 78 60 95



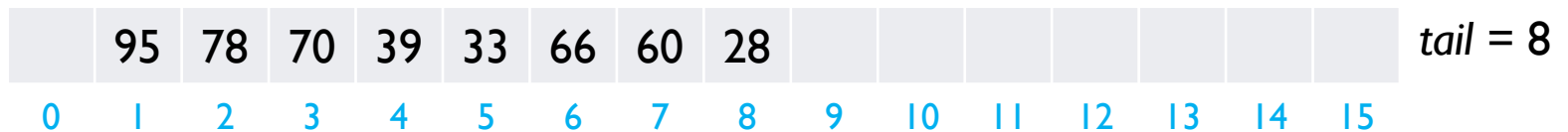
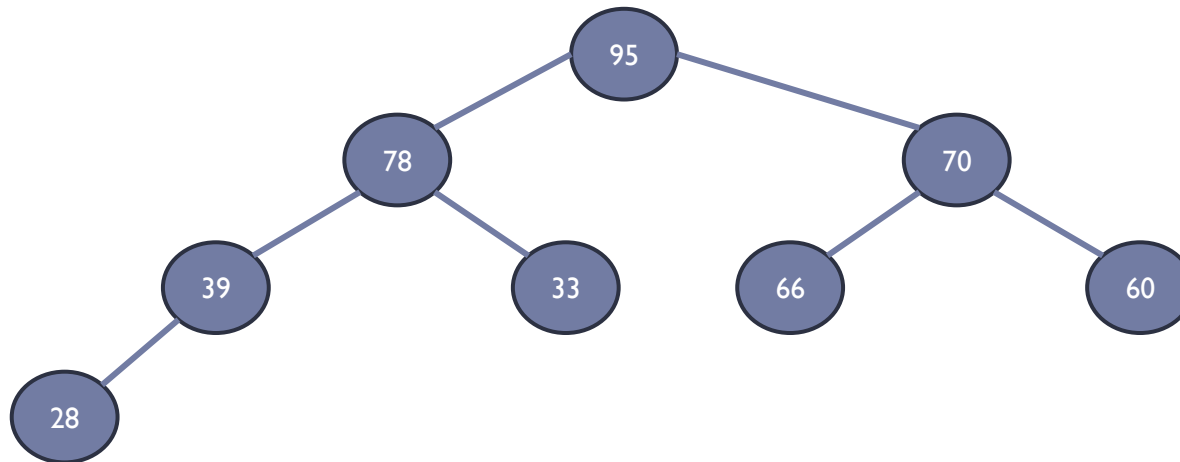


Árvores HEAP

TAD - Inserção

► Considere os elementos:

► 33 70 66 28 39 78 60 95



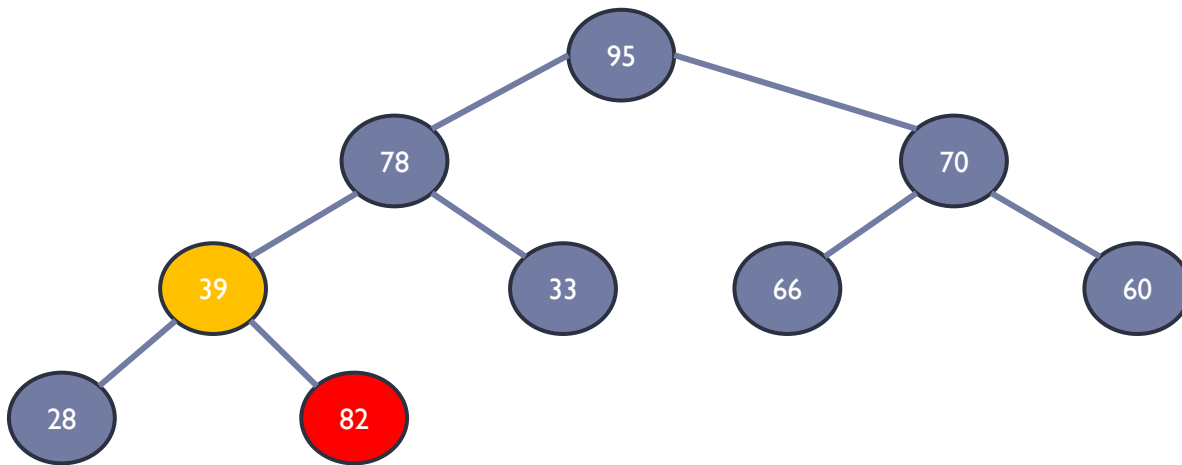


Árvores HEAP

TAD - Inserção

► Considere os elementos:

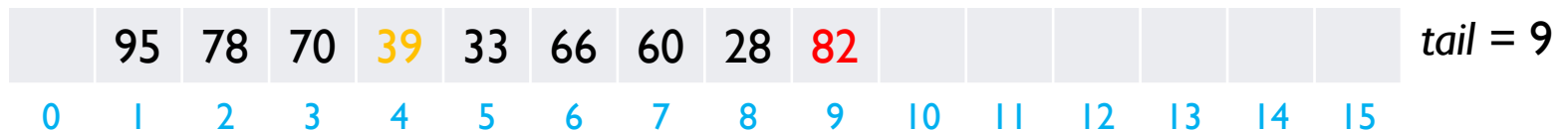
► 33 70 66 28 39 78 60 95



Inserir o nó 82



- Insere na posição $tail + 1$
- Atualiza o tail
- Verifica se o nó é maior que o pai
- Enquanto for maior que o pai e não for raiz, troca.



$$pai = \lfloor 9/2 \rfloor \Rightarrow 4$$

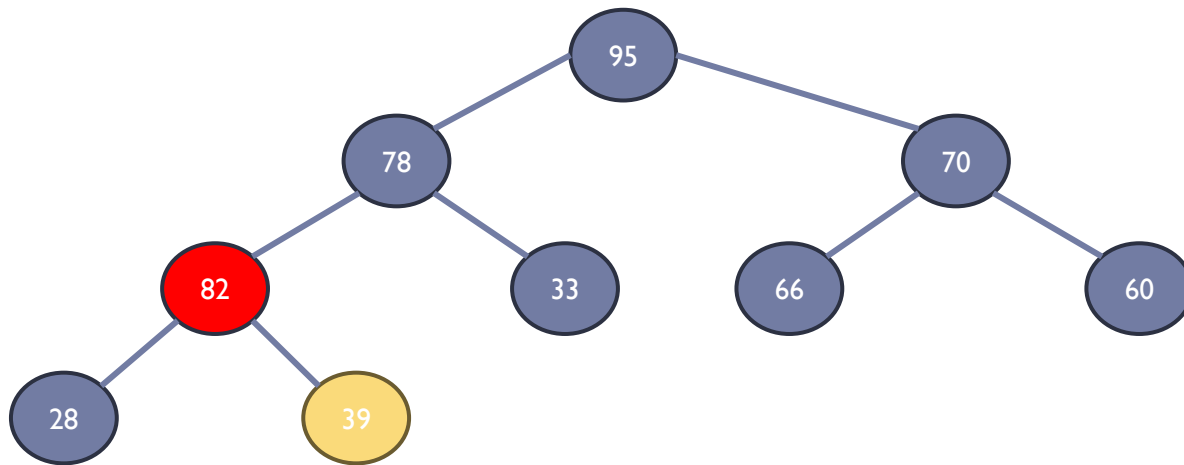


Árvores HEAP

TAD - Inserção

► Considere os elementos:

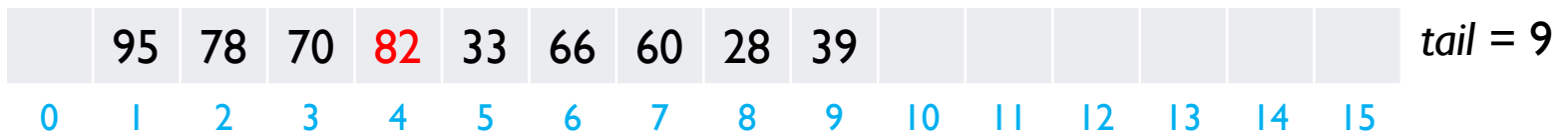
► 33 70 66 28 39 78 60 95



Inserir o nó 82



- Insere na posição *tail + 1*
- Atualiza o *tail*
- Verifica se o nó é maior que o pai
- Enquanto for maior que o pai e não for raiz, troca.



82 > 39 => TROCA

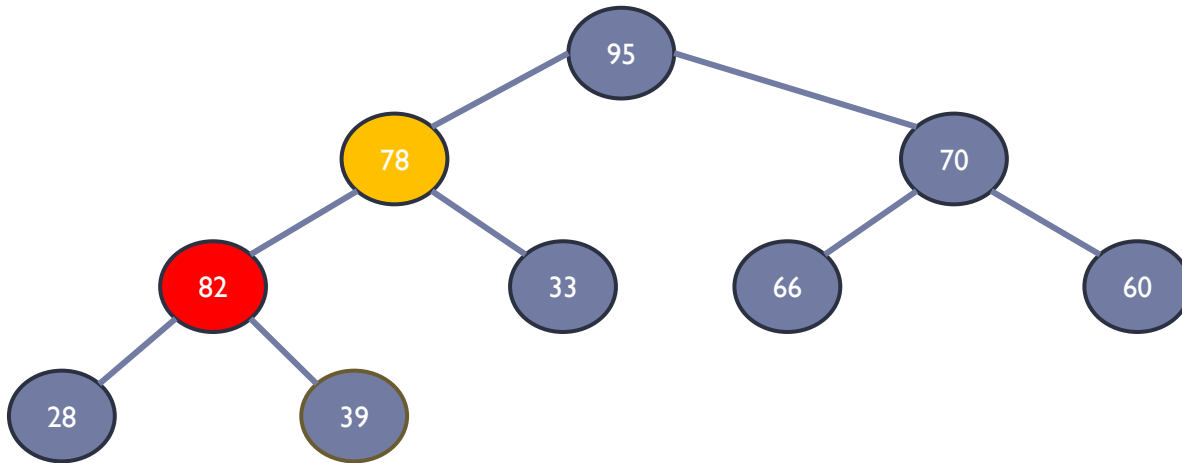


Árvores HEAP

TAD - Inserção

► Considere os elementos:

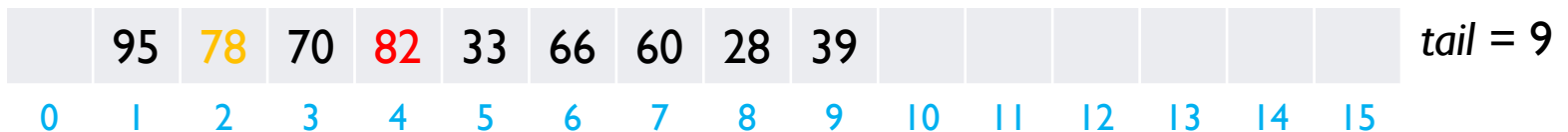
► 33 70 66 28 39 78 60 95



Inserir o nó 82



- Insere na posição $tail + 1$
- Atualiza o tail
- Verifica se o nó é maior que o pai
- Enquanto for maior que o pai e não for raiz, troca.



$$pai = \lfloor 4/2 \rfloor \Rightarrow 2$$

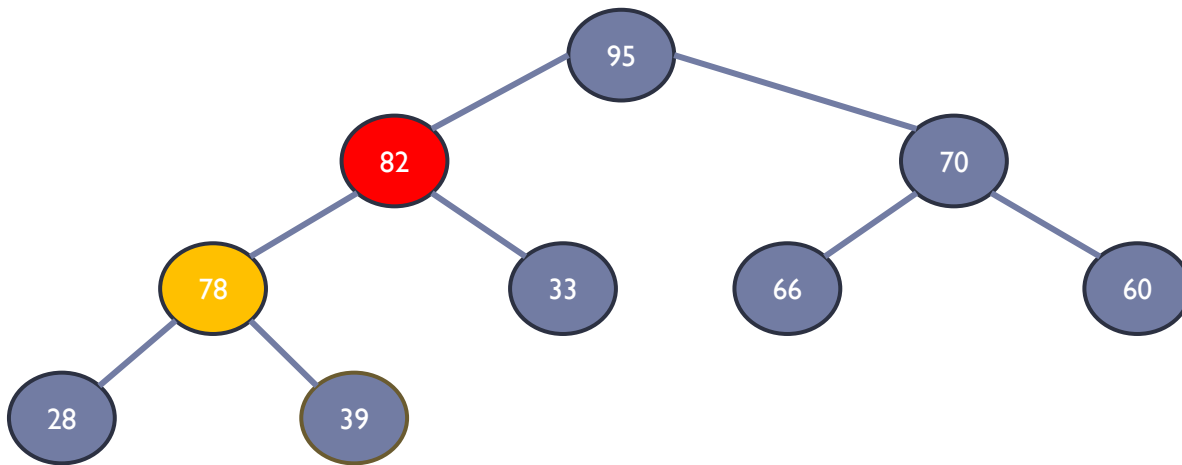


Árvores HEAP

TAD - Inserção

► Considere os elementos:

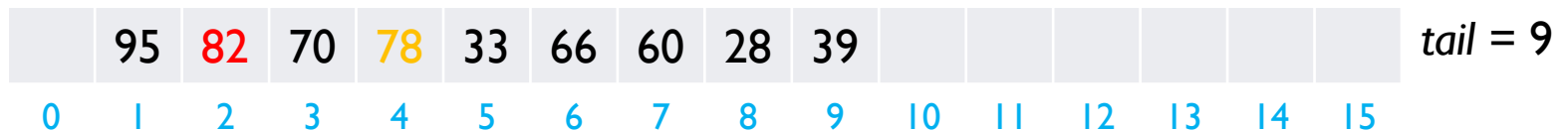
► 33 70 66 28 39 78 60 95



Inserir o nó 82



- Insere na posição $tail + 1$
- Atualiza o tail
- Verifica se o nó é maior que o pai
- Enquanto for maior que o pai e não for raiz, troca.



$82 > 78 \Rightarrow \text{TROCA}$

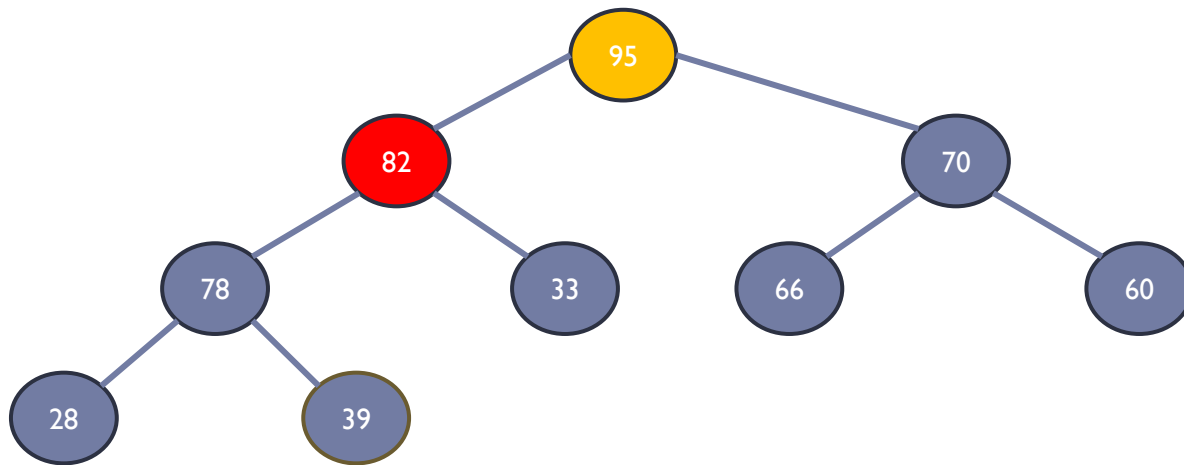


Árvores HEAP

TAD - Inserção

► Considere os elementos:

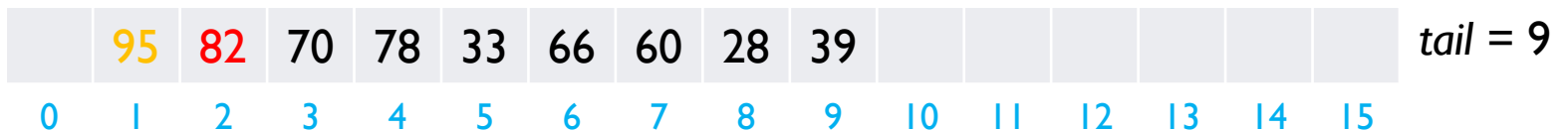
► 33 70 66 28 39 78 60 95



Inserir o nó 82



- Insere na posição *tail + 1*
- Atualiza o tail
- Verifica se o nó é maior que o pai
- Enquanto for maior que o pai e não for raiz, troca.



$$pai = \lfloor \frac{2}{2} \rfloor \Rightarrow 1$$

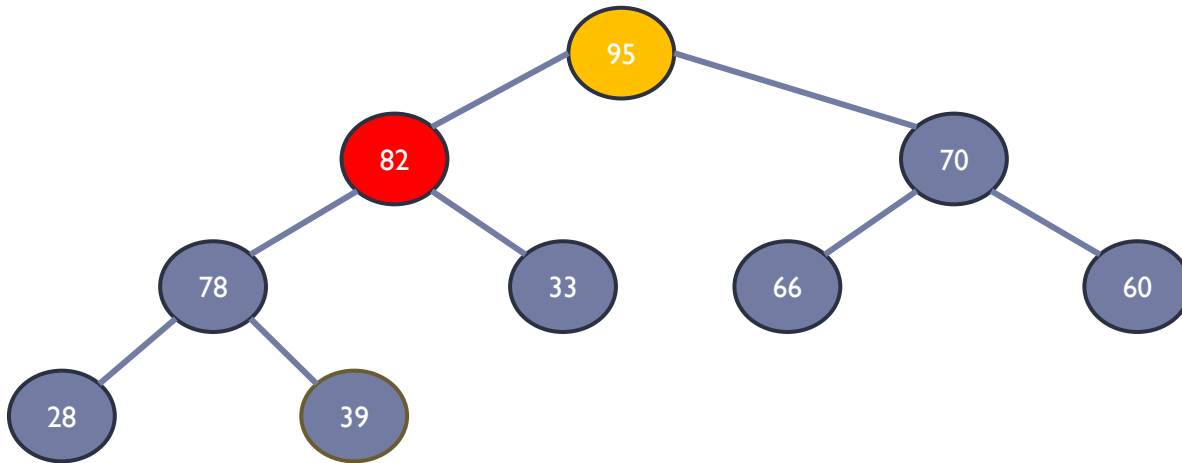


Árvores HEAP

TAD - Inserção

► Considere os elementos:

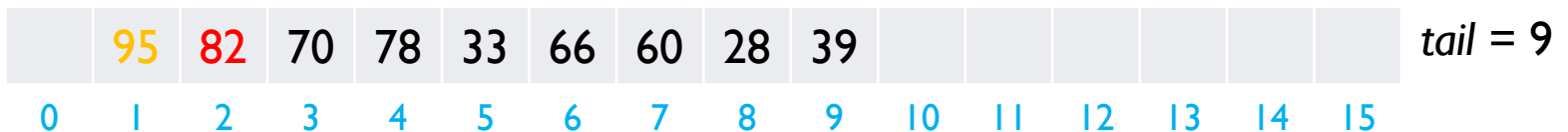
► 33 70 66 28 39 78 60 95



Inserir o nó 82



- Insere na posição *tail + 1*
- Atualiza o *tail*
- Verifica se o nó é maior que o pai
- Enquanto for maior que o pai e não for raiz, troca.



82 > 95 => ENCERRA

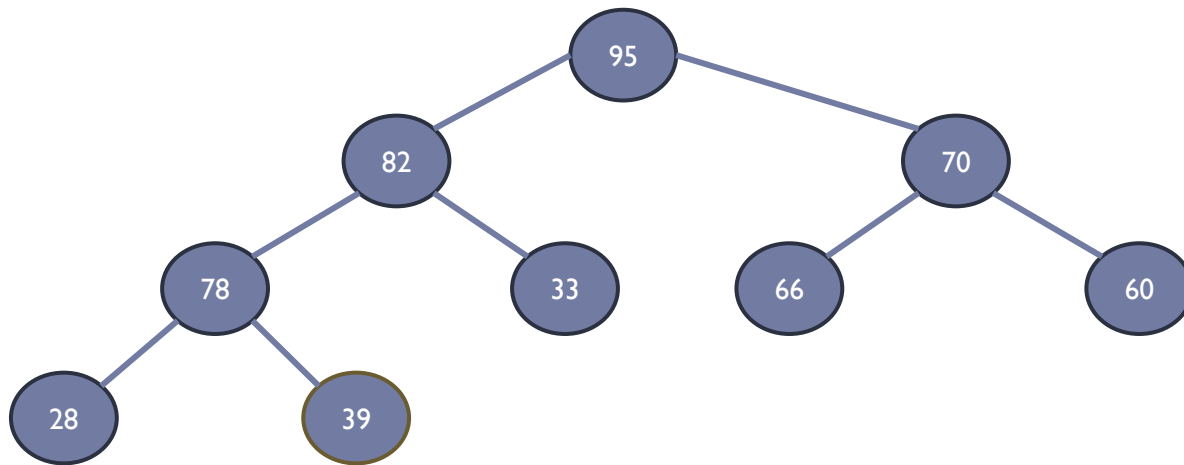


Árvores HEAP

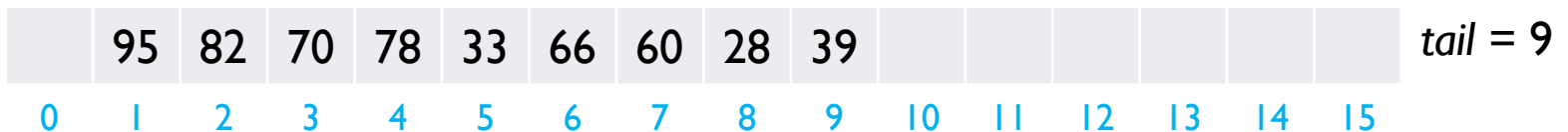
TAD - Inserção

► Considere os elementos:

► 33 70 66 28 39 78 60 95



- Insere na posição $tail + 1$
- Atualiza o tail
- Verifica se o nó é maior que o pai
- Enquanto for maior que o pai e não for raiz, troca.





► Exercício

- Considere os valores 12, 25, 13, 17, 19, 3, 14, 5, 11
 - a) Simule a entrada dos valores em um HEAP Máximo
 - i. Após HEAP pronto, insira os valores 30 e 42
 - b) Simule a entrada dos valores em um HEAP Mínimo
 - i. Após o HEAP pronto, insira os valores 18 e 2



Implementação

Geração e Inserção



Árvores HEAP

Implementação – Geração e Inserção

- ▶ **heap *criaHeap(int tam);**
 - ▶ tam indica a quantidade máxima de elementos da heap
- ▶ **int insereElemento(heap *h, int elemento);**
 - ▶ Considera um heap máximo
- ▶ **int carregaElementos(char nomeArquivo[], heap *h);**
 - ▶ Lê os dados do arquivo e gera a heap
- ▶ **int retornaOcupacao(heap *h);**
 - ▶ Retorna o total de elementos na heap
- ▶ **int retornaMaiorPrioridade(heap *h);**
 - ▶ Retorna a raiz da heap, mas não remove
- ▶ **void imprimeVetor(heap *h);**
 - ▶ Para fins de verificação

```
struct heap
{
    int *vet;
    int tail;
    int tam;
};
```



A vertical blue bar is located on the left side of the slide, partially enclosed by a thin white rectangular border.

TAD HEAP

REMOÇÃO



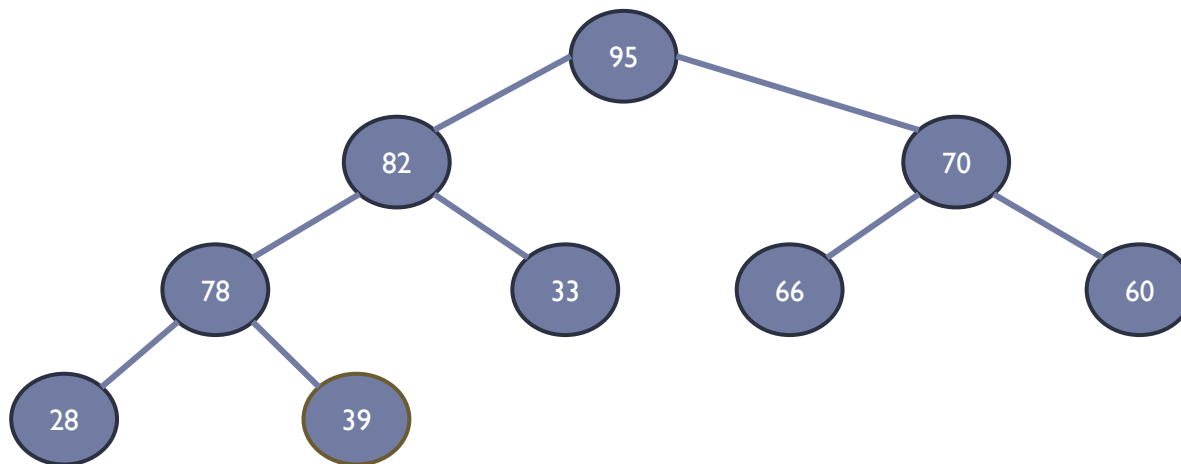
- ▶ HEAP representa uma fila de prioridade em árvore binária.
- ▶ No conceito de FILA, todo elemento deve ser removido do começo da fila
 - ▶ Na árvore binária heap, o começo da fila é a raiz



Árvores HEAP

TAD - Remoção

- ▶ Como remover a raiz?
- ▶ Como garantir a propriedade de max-heap?





Árvores HEAP

TAD - Remoção

- ▶ Existe um algoritmo chamado HEAPIFY, que faz a remoção da raiz da árvore e garante a ordenação.
- ▶ Esse algoritmo troca a raiz pelo elemento de menor prioridade
 - ▶ Copia o último elemento do vetor para a primeira posição
 - ▶ Inutiliza a última posição
 - ▶ Como??
 - ▶ Posteriormente corrige a heap
 - ▶ Compara o pai e seus dois filhos.
 - Se o pai for menor que os filhos, o maior filho troca de lugar com o pai e o processo se repete até que o pai seja folha ou não seja menor que os filhos

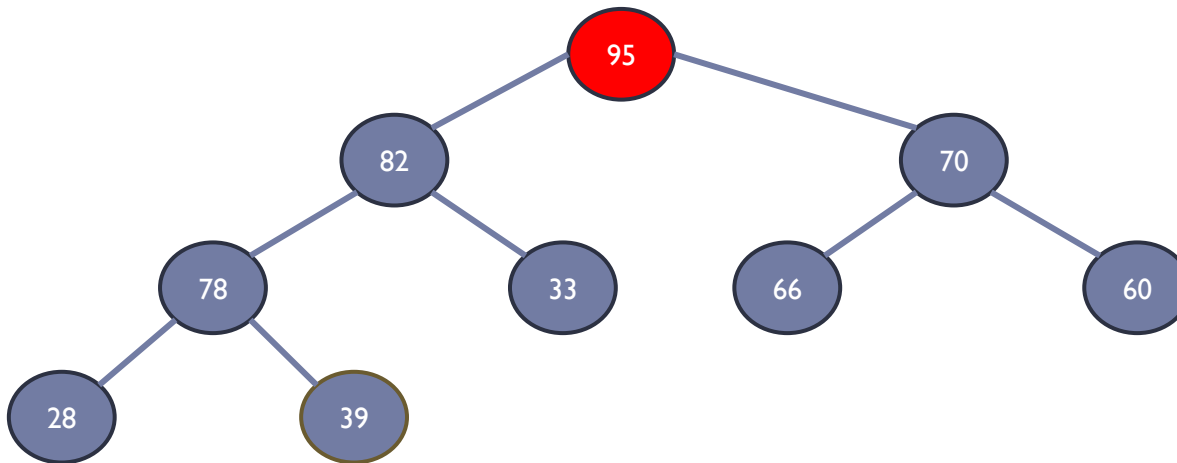




Árvores HEAP

TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



	95	82	70	78	33	66	60	28	39							<i>tail = 9</i>
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	

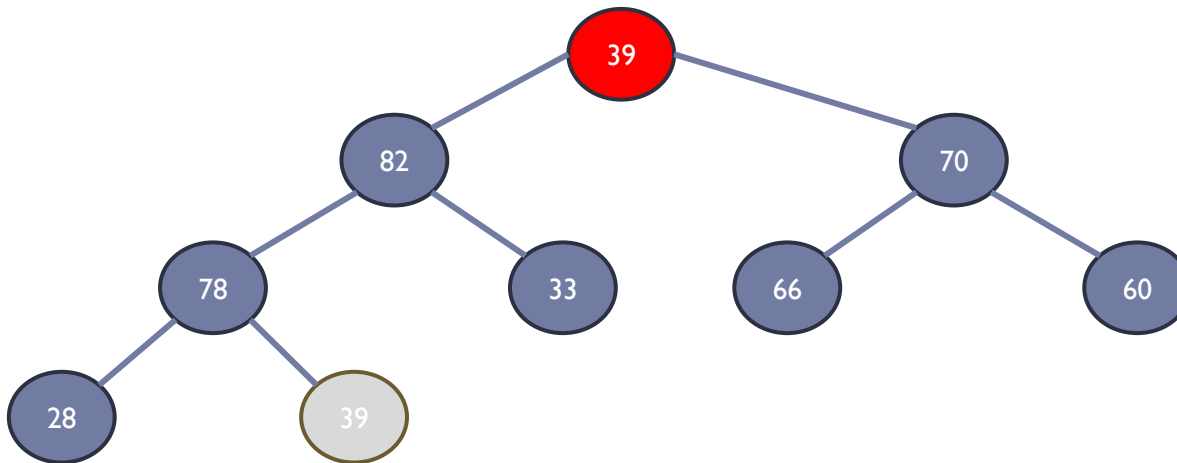




Árvores HEAP

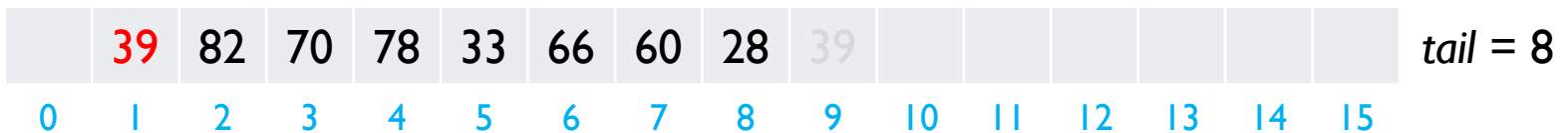
TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



Troca a raiz com o elemento de menor prioridade

Inutiliza a última posição

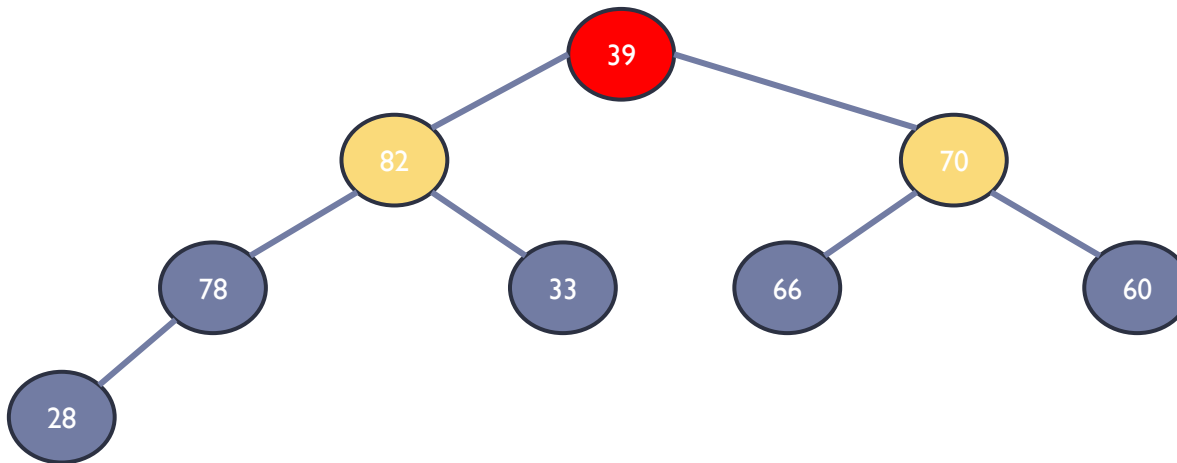




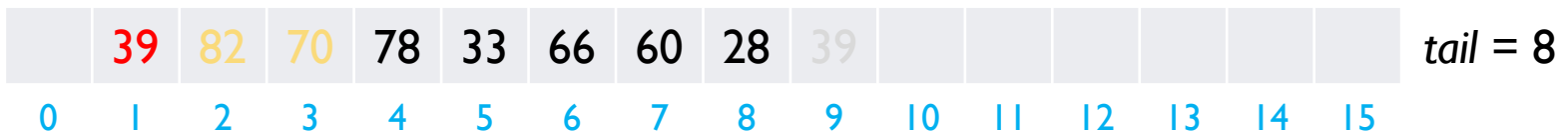
Árvores HEAP

TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



*Compara o pai e os filhos.
Qual é o maior?
Se for um dos filhos, troca e continua.*

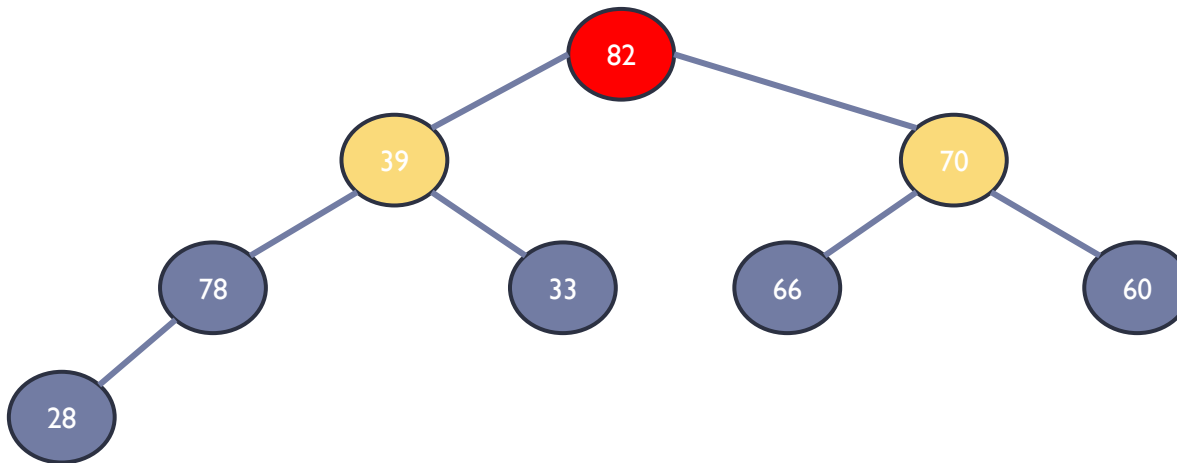




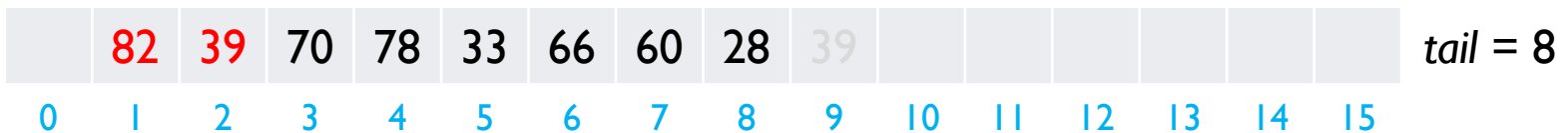
Árvores HEAP

TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



Compara o pai e os 2 filhos. Qual é o maior? Se for um dos filhos, troca e continua.

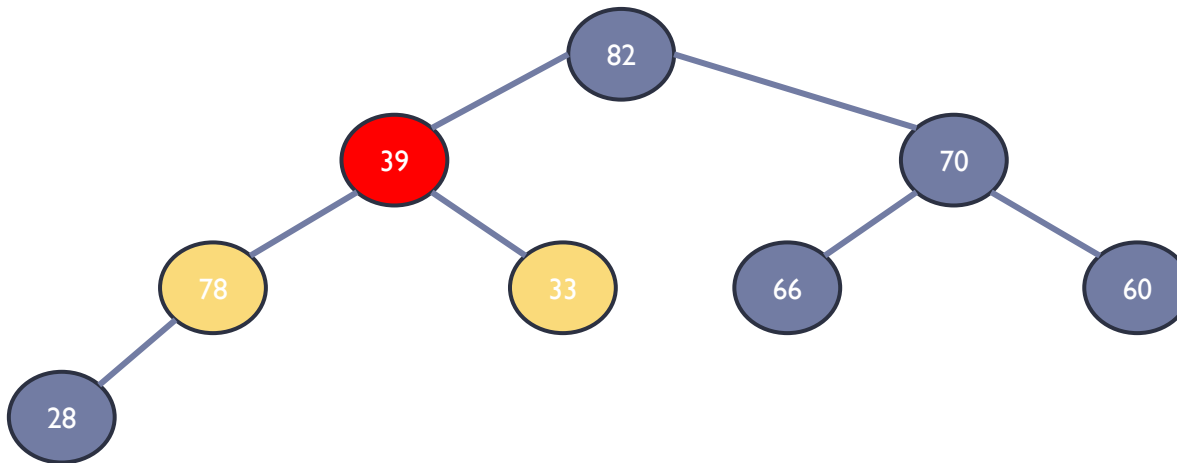




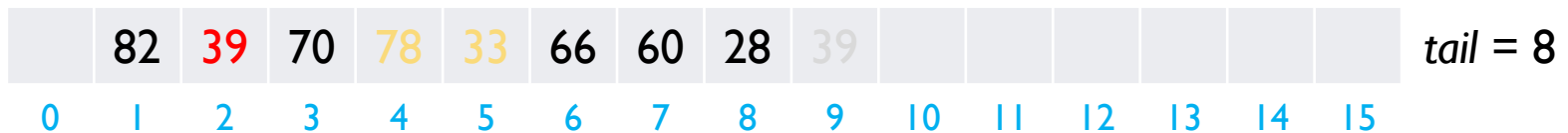
Árvores HEAP

TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



*Compara o pai e os filhos.
Qual é o maior?
Se for um dos filhos, troca e continua.*

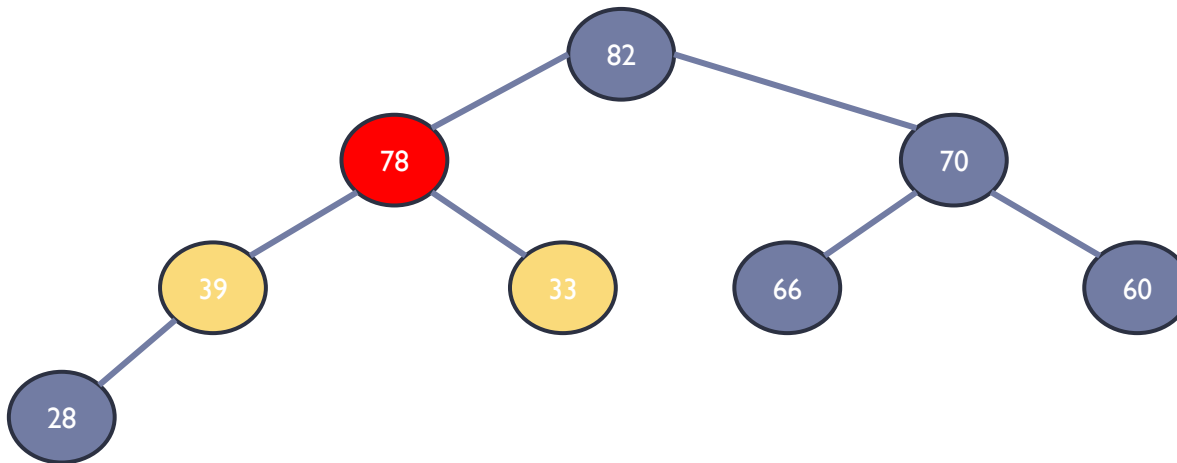




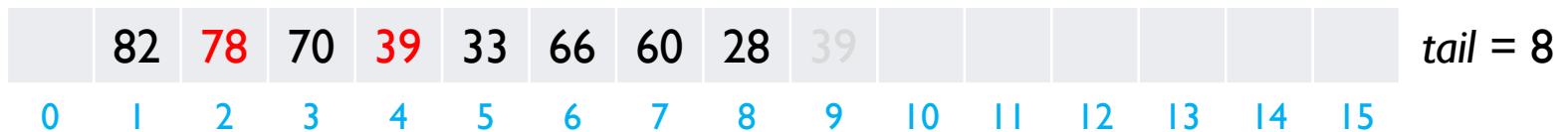
Árvores HEAP

TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



*Compara o pai e os filhos.
Qual é o maior?
Se for um dos filhos, troca e continua.*

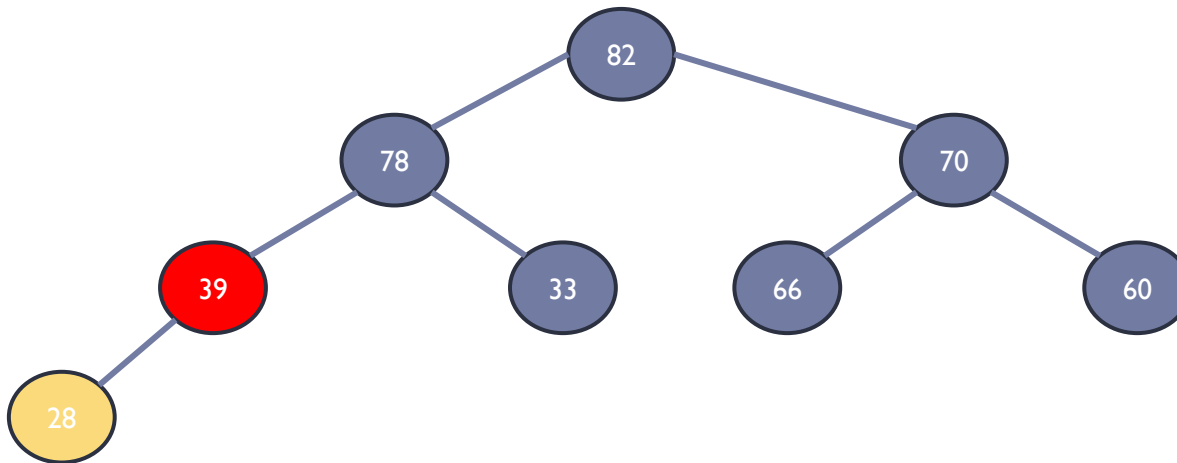




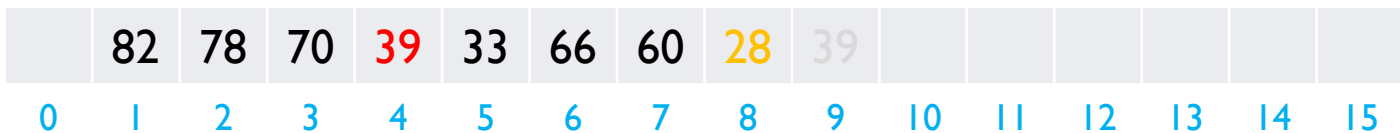
Árvores HEAP

TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



*Compara o pai e os filhos.
Qual é o maior?
Se for um dos filhos, troca e continua.*



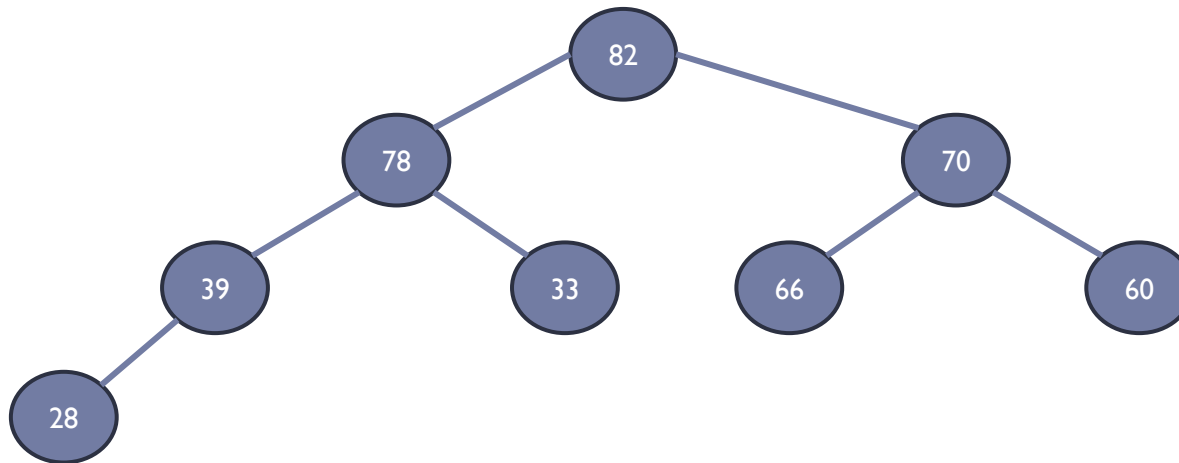
tail = 8



Árvores HEAP

TAD - Remoção

- ▶ Remover da HEAP de máximo abaixo
 - ▶ O elemento removido será o 95



Remoção
finalizada



tail = 8

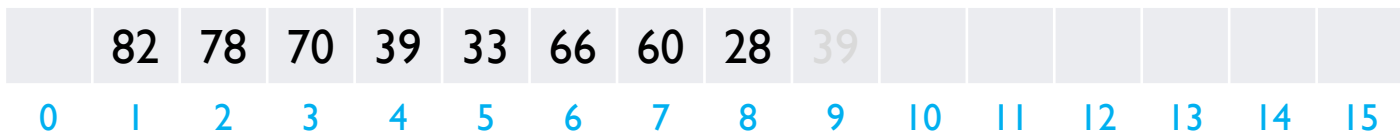
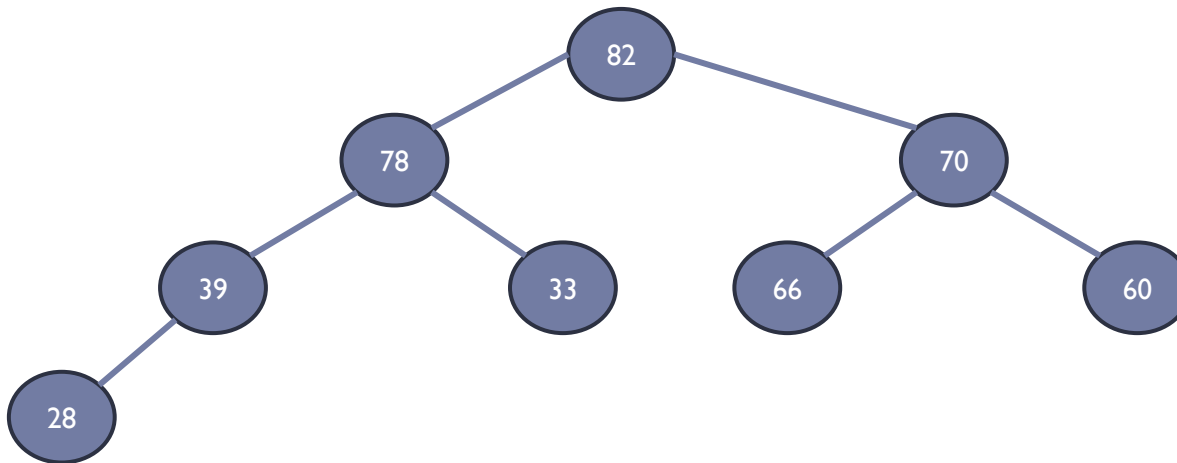


Árvores HEAP

TAD - Remoção

▶ Exercício

- ▶ Remover um elemento do HEAP abaixo



tail = 8

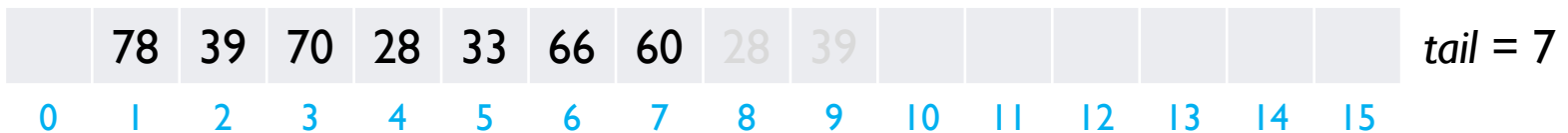
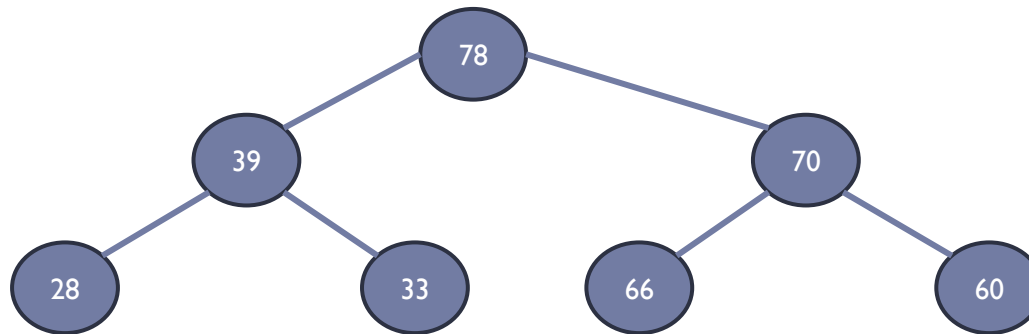


Árvores HEAP

TAD - Remoção

► Exercício

- Remover um elemento do HEAP abaixo

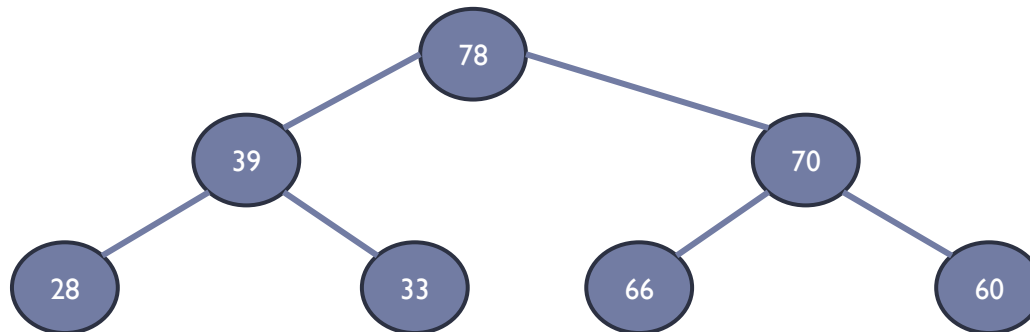




Árvores HEAP

TAD – Alteração de Prioridade

- ▶ O HEAPIFY também é utilizado em uma operação típica do HEAP que é a mudança na prioridade de um elemento.
- ▶ Na árvore abaixo, suponha que o nó 70 tenha tido sua prioridade atualizada para 20.
 - ▶ Isso quebraria a propriedade do max-heap
 - ▶ Portanto, aplica-se o Heapify para restaurar a propriedade da árvore

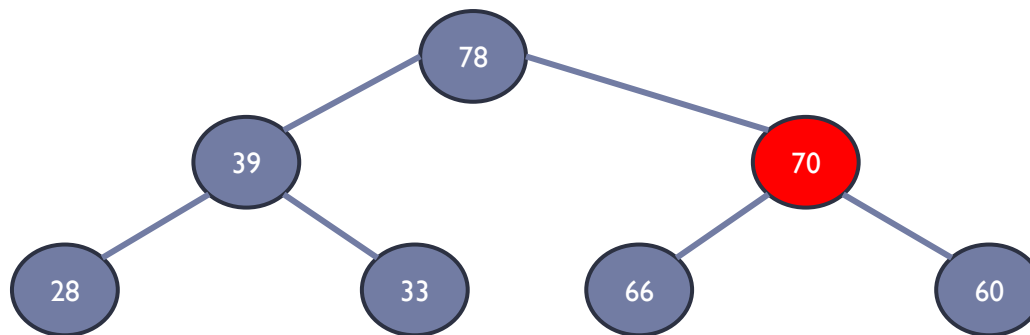




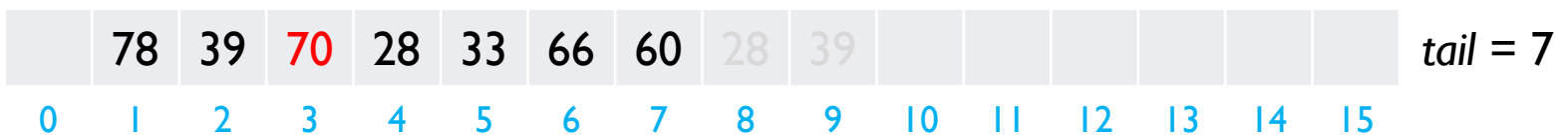
Árvores HEAP

TAD – Alteração de Prioridade

- ▶ Na árvore abaixo, suponha que o nó 70 tenha tido sua prioridade atualizada para 20.
 - ▶ Isso quebraria a propriedade do max-heap
 - ▶ Portanto, aplica-se o Heapify para restaurar a propriedade da árvore



Altera a prioridade do nó

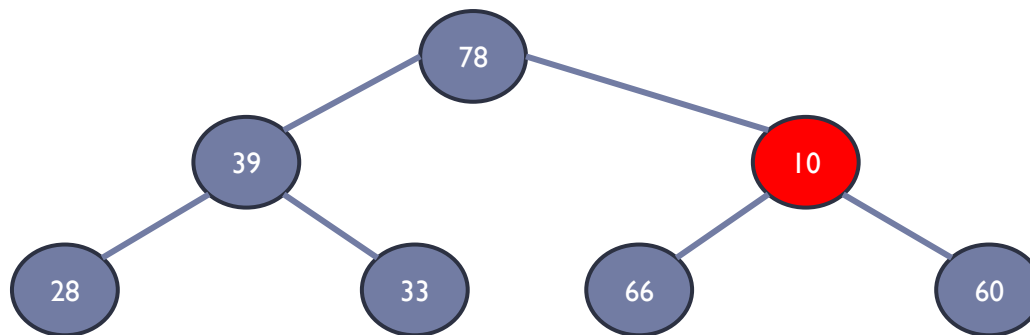




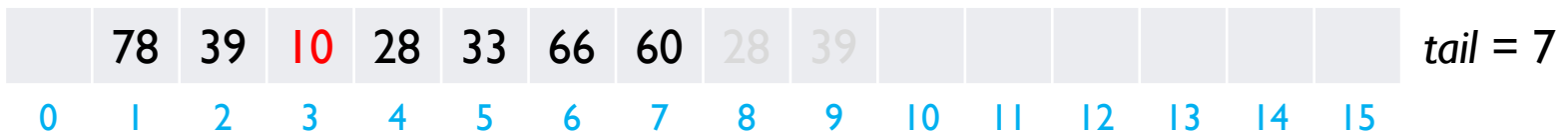
Árvores HEAP

TAD – Alteração de Prioridade

- ▶ Na árvore abaixo, suponha que o nó 70 tenha tido sua prioridade atualizada para 20.
- ▶ Isso quebraria a propriedade do max-heap
- ▶ Portanto, aplica-se o Heapify para restaurar a propriedade da árvore



Aplica o Heapify a partir do nó alterado

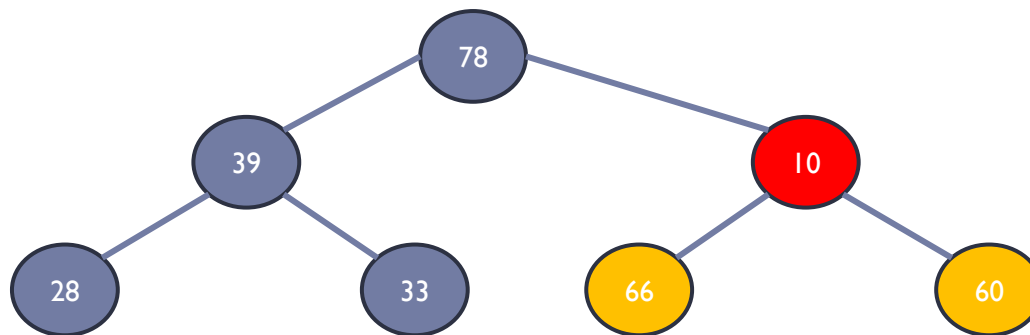




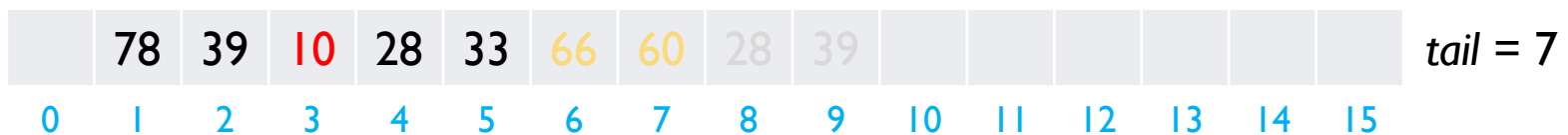
Árvores HEAP

TAD – Alteração de Prioridade

- ▶ Na árvore abaixo, suponha que o nó 70 tenha tido sua prioridade atualizada para 20.
- ▶ Isso quebraria a propriedade do max-heap
- ▶ Portanto, aplica-se o Heapify para restaurar a propriedade da árvore



*Compara o pai e os filhos.
Qual é o maior?
Se for um dos filhos, troca e continua.*

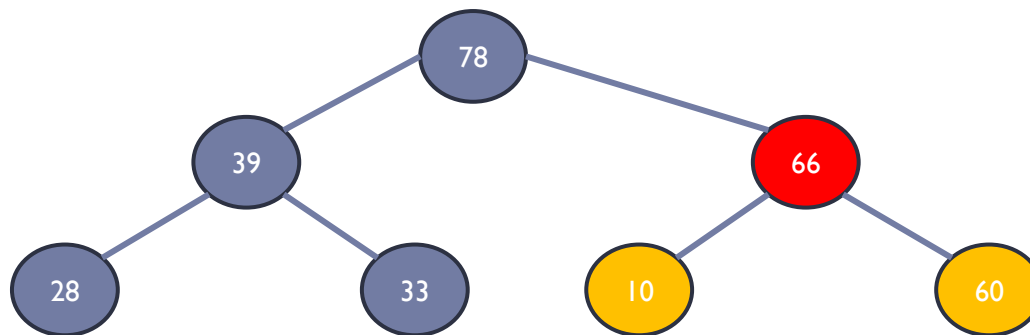




Árvores HEAP

TAD – Alteração de Prioridade

- ▶ Na árvore abaixo, suponha que o nó 70 tenha tido sua prioridade atualizada para 20.
- ▶ Isso quebraria a propriedade do max-heap
- ▶ Portanto, aplica-se o Heapify para restaurar a propriedade da árvore



*Compara o pai e os filhos.
Qual é o maior?
Se for um dos filhos, troca e continua.*

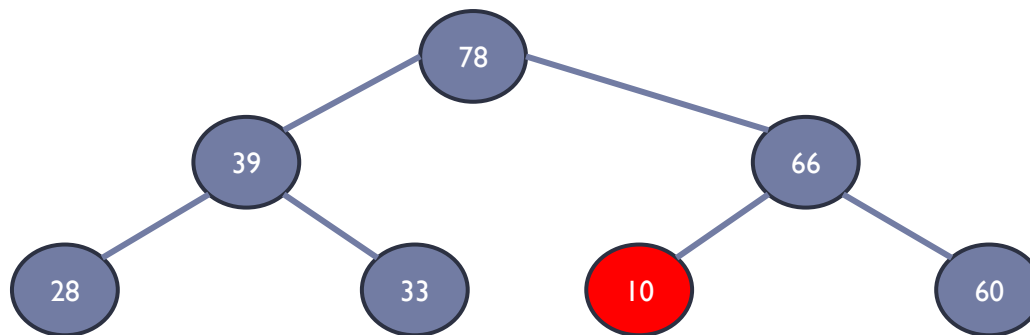




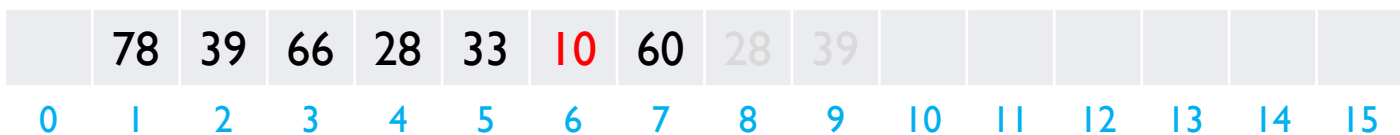
Árvores HEAP

TAD – Alteração de Prioridade

- ▶ Na árvore abaixo, suponha que o nó 70 tenha tido sua prioridade atualizada para 20.
 - ▶ Isso quebraria a propriedade do max-heap
 - ▶ Portanto, aplica-se o Heapify para restaurar a propriedade da árvore



Como 10 agora é folha, finaliza



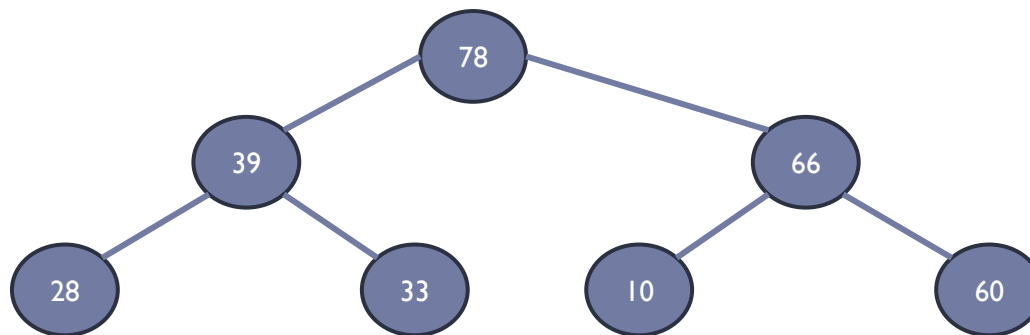
tail = 7



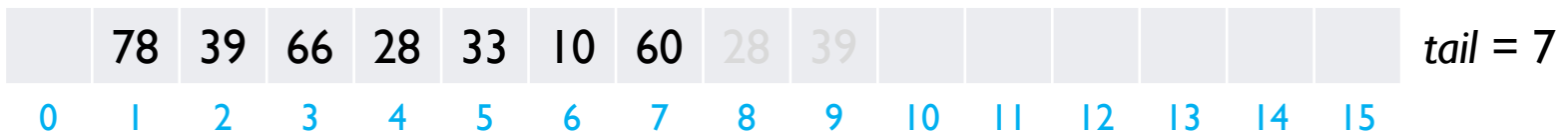
Árvores HEAP

TAD – Alteração de Prioridade

- ▶ Na árvore abaixo, suponha que o nó 70 tenha tido sua prioridade atualizada para 20.
- ▶ Isso quebraria a propriedade do max-heap
- ▶ Portanto, aplica-se o Heapify para restaurar a propriedade da árvore



*Alteração de
prioridade
finalizada*





Implementação

Remoção

Alteração de Prioridade



Implementação – Remoção

► `int removeElemento(heap *h);`

- Remove um elemento da heap
- Retorna -1 caso a heap esteja vazia
- Retorna o elemento removido, caso contrário

► `void MaxHeapify(heap *h, int pos);`

- Realiza a correção da heap após uma remoção ou alteração de prioridade
- Pseudocódigo no livro do Cormen - 2 edição - pag. 76

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $largest = l$ 
5  else  $largest = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[largest]$ 
7       $largest = r$ 
8  if  $largest \neq i$ 
9      exchange  $A[i]$  with  $A[largest]$ 
10     MAX-HEAPIFY( $A, largest$ )
```